

What is control plane in architecture above?

What is control plane in architecture above?

[What is control plane in architecture above?](#)

[Quick glossary \(skim this first\)](#)

[The core idea in one line](#)

[Why we're doing it this way](#)

[The three pieces](#)

[Piece 1 — Policy authoring \(writing the rules\)](#)

[Piece 2 — Resolver \(combining the layers\)](#)

[Piece 3 — Distribution \(getting rules to the gateway\)](#)

[What we're actually building](#)

[The demo \(under 2 minutes\)](#)

[Open questions — please weigh in](#)

Quick glossary (skim this first)

Term	What it means
Gateway	The piece of code that sits between the user and the AI model. Every request and response passes through it. It's what actually runs our safety checks.
Data plane	All the parts that handle <i>live traffic</i> — the gateway, the detectors, the decision to allow/block. Busy, fast, running constantly.
Control plane	The parts that decide <i>what the rules should be</i> . Slow, human-driven, changes rarely.
Policy	Our safety rules written down as settings — e.g. "redact anything that looks 80% likely to be a phone number."

Term	What it means
Detector	One specific check. We have several: a PII detector, a grounding detector, etc.
PII	Personally Identifiable Information — names, phone numbers, IDs, addresses. Stuff we shouldn't leak.
Grounding	Whether the AI's answer is actually supported by the source documents, or made up. Low grounding = likely hallucination.
Threshold	The confidence level at which a detector acts. A detector outputs a score 0–1; the threshold is where we draw the line.
Bundle	One finished, flat file of settings that the gateway reads. Produced by combining our layered policy files.
Audit ledger	A log where we record every decision the system made and why.

The core idea in one line

Control plane = deciding the rules. Data plane = applying the rules to a specific request.

We keep them separate so we can change our safety rules without touching or redeploying code.

Why we're doing it this way

We're guarding three different AI products at once:

1. A **customer support bot** — talks to real customers, must be fast
2. An **internal knowledge assistant** — for employees, more forgiving
3. A **decision-support tool** used in a regulated workflow — slowest, strictest

All three need safety checks, but *different* ones. The support bot can't afford a slow check because a customer is waiting. The regulated tool can afford it and legally needs it.

If we hardcode those differences into the checking code, three bad things happen:

- Changing one number for one product means a code deploy that could break the other two

- If an auditor asks "what rule was live on 14 March, and who approved it?" we're digging through git history
- Rules go stale, because updating them feels risky

So our approach: **the rules are data, not code**. Humans edit simple config files. A small program turns them into settings. The gateway just does what the settings say.

The three pieces

Piece 1 — Policy authoring (writing the rules)

A person — ideally a risk/compliance owner, not necessarily an engineer — writes rules in YAML. (**YAML** is just a plain-text format for settings. It's `key: value` lines, no programming needed.)

We write them in **layers**, from general to specific:

yaml

```
# org-baseline.yaml – applies to everything we build
pii_threshold: 0.8
on_pii: redact
# jurisdiction-eu.yaml – extra rules because EU law is stricter
pii_threshold: 0.5    # locked
grounding_threshold: 0.6
# support-bot.yaml – settings just for the support bot
pii_threshold: 0.7
latency_budget_ms: 200
```

Reading that: `pii_threshold: 0.8` means "if the PII detector is 80% or more confident it found personal information, act on it." `on_pii: redact` means the action is to black it out rather than block the whole response. `latency_budget_ms: 200` means the support bot's checks must finish within 200 milliseconds.

Why layers? So we don't copy-paste the same compliance rules into twenty files. Update EU law once, everything inheriting it updates.

Piece 2 — Resolver (combining the layers)

The gateway can't read three overlapping files. It needs **one flat list** of final settings. The resolver is the small program that squashes the layers into that one list.

Nothing magic here — it's a merge function we write ourselves, maybe 40 lines.

The interesting question: the three files above all set `pii_threshold`, to 0.8, 0.5, and 0.7. Which wins?

Our rule: field-level locking.

- A compliance layer can mark a setting as **locked**
- Inner layers may make a locked setting *stricter*, never *looser*
- Settings that aren't locked just take the most specific layer's value (normal behaviour)

Working through it: EU set `pii_threshold: 0.5` and locked it. Support-bot asked for 0.7.

Here's the bit that trips people up — **lower is stricter for a threshold**. At 0.5 we act on weaker evidence, so we catch *more* things. At 0.7 we need stronger evidence, so we catch *fewer*. So 0.7 is a request to be more lenient than EU allows → it gets clamped back to 0.5.

Final bundle handed to the gateway:

yaml

```
pii_threshold: 0.5
on_pii: redact
grounding_threshold: 0.6
latency_budget_ms: 200
```

Notice support-bot's file wasn't ignored — it still supplied `latency_budget_ms`. Only the single field where it tried to be more lenient got overridden. And if it had asked for 0.3 (stricter than 0.5), that would have been allowed.

⚠️ **We never clamp silently.** The resolver prints a warning: `support-bot.yaml`
`requested pii_threshold 0.9, clamped to 0.5 by jurisdiction-eu (locked)` — and that goes in the audit log. So if a team keeps pushing at a boundary, it's visible to everyone.

Why locking is the point (use this line in the pitch): the team running the support bot is measured on customer satisfaction. Over-redacting annoys customers and hurts their numbers. So loosening a threshold is a *sensible* move from where they sit. Locking means our compliance rules can't quietly stop applying just because someone had a good local reason.

One wrinkle: "stricter" points in different directions for different settings.

- Lower `pii_threshold` = stricter (catches more)
- Higher `latency_budget_ms` = stricter (more time = we can run more checks)

So the resolver keeps a small lookup table saying which direction is strict for each setting.

The resolver also sanity-checks. If a file says "finish in 200ms" but switches on a check that takes 600ms, the resolver refuses to build the bundle. Better to fail on our laptops than time out on every live request.

Piece 3 — Distribution (getting rules to the gateway)

The bundle exists. Now it has to reach the gateway handling real traffic. Three jobs:

a) Version and hash it.

Each bundle gets stamped: `support-bot-eu, v7, sha256:a3f9c2...`. (**Hash** = a short fingerprint calculated from the file's contents. Change one character, you get a totally different fingerprint.)

Every decision the gateway makes gets logged along with that fingerprint. So months later, "why was this blocked in March?" is exactly answerable — look up the record, read the fingerprint, pull up that exact bundle. Without it, we'd be guessing.

b) Shadow deploy.

We changed a threshold and don't know what it'll do to real traffic. So we run the old version (v6) and new version (v7) side by side — but only v6's decisions are actually enforced. v7 just records what it *would* have done.

After a day: "v7 would have blocked 340 responses that v6 allowed." Now we can judge whether that's catching real problems or would have broken the product — before any user is affected.

📌 **Don't mix this up with "shadow eval"** in the learning plane. Shadow eval samples a bit of traffic and runs our expensive deep check on it, to estimate how many bad things we're *missing*. Shadow deploy compares two versions of our *rules*. Different things, similar name.

c) Rollback.

v7 goes live and starts flagging everything. We revert to v6 in seconds. No code deploy, no engineer needed. That's the whole payoff of separating rules from code.

What we're actually building

Real, working code:

- ☐ 3 YAML layer files
- ☐ `resolve()` — the merge with locking, ~40 lines
- ☐ `compile()` — resolve, add version number, hash it, save to `bundles/`, ~20 lines
- ☐ Gateway loads a bundle by name and enforces it
- ☐ Every decision writes a row to the audit ledger: `{request_id, timestamp, policy_name, policy_version, policy_hash, detector_scores, action}`

Stubbed (simplified) — and we say so openly in the deck:

- **Shadow deploy** → a script that runs 50 saved test requests through v6 and v7 and prints a comparison table. Not live traffic, but it demonstrates the mechanism fully.
- **Rollback** → literally one line: `compile('support-bot', pin='v6')`
- **Authoring console** (a web UI for editing policies) → **skipping entirely**. Editing a YAML file live on stage is more convincing than a form anyway.

Being upfront about what's built vs. simplified vs. only designed reads as maturity to judges. Hiding it reads as a gap when they probe.

The demo (under 2 minutes)

1. **Same prompt, two bundles.** Support bot redacts and answers. Decision-support escalates and blocks. Zero code difference — we only changed which bundle was loaded. This is the whole idea, proven in one step.
2. **Try to break the rules.** Set `pii_threshold: 0.9` in support-bot.yaml, recompile. Resolver prints the clamp warning. Behaviour doesn't change. Governance, proven.
3. **Change something that isn't locked.** Recompile → v8, new fingerprint. Re-run the prompt, decision flips.
4. **Shadow diff.** Run v7 vs v8 over the 50 saved requests, show how many decisions changed.
5. **Open the ledger.** Three records, three different policy fingerprints.

That covers five things judges reliably ask about: separating policy from code, enforcing governance, versioning, safe rollout, and auditability.

Open questions — please weigh in

- **Which settings do we lock** in the EU layer? *Proposal: PII and grounding thresholds only, nothing else.*
- **If the checker itself times out, do we fail open or closed?** (Fail open = let the response through unchecked. Fail closed = block it.) *Proposal: internal copilot fails open with a visible warning banner; decision-support fails closed.*
- **Who "owns" a policy in our fictional company, and who signs off on a version bump?** We need names/roles for the governance slide.